Design Automation Platform — Realistic End-to-End Industry POC Story

> **Purpose of this document**
>
> This is a reconstructed, industry-realistic story of how the Design Automation Platform could have progressed inside an IT services company working for a client. It uses the **final technical architecture and measured results from the hands-on reconstruction** as the technical source of truth. Organizational details such as team composition, sprint timing, meetings, and stakeholder roles are conservative assumptions added to make the project understandable as a real consulting engagement.
>
> It should be used to build a natural interview narrative, not to claim that every reconstructed implementation detail is a verified historical fact from the original project.

---

1. Project setup at a glance

The client was a large retail/e-commerce organization with a mature Order Management System and several downstream services. Their engineering teams regularly produced Solution Design Documents (SDDs) before implementing changes. For significant features, a solution architect or senior engineer had to search through requirements, architecture documents, API contracts, older design documents, and system-dependency information before writing an impact analysis.

The client was not asking us to replace architects. Their problem was that a large amount of time was spent on the **first-pass discovery and documentation work**. Two engineers could read the same set of documents and still produce drafts with different levels of completeness. Existing information was scattered across documents and teams, and technical patterns from earlier designs were frequently rediscovered manually.

Our services team proposed a POC for a **grounded design-automation assistant**. The POC would take a business/technical requirement, retrieve relevant enterprise evidence, enrich it with system relationships, and generate a traceable draft of one SDD section—primarily an **impact analysis**—for architect review.

Realistic POC duration

A credible duration for this scope is **12 weeks**.

That is long enough to cover discovery, data preparation, custom retrieval, graph enrichment, generation, API integration, operational hardening, and evaluation, while remaining small enough to still be a POC rather than a full production program.

The approximate calendar would be:

```
Weeks 1–2    Discovery, sample corpus, scope, architecture
Weeks 3–4    Ingestion + dense retrieval baseline
Weeks 5–6    BM25 + hybrid retrieval + graph enrichment
```

```
Weeks 7–8    Grounded context + prompts + LLM generation + persistence
Weeks 9–10   FastAPI + reliability + failure handling
Weeks 11–12  Evaluation, tuning, final client demo, production
recommendations
```

The project ran in five two-week engineering sprints after a two-week discovery phase.

Team size

The team was intentionally small. There were roughly **five people contributing regularly**, with three additional roles shared part-time.

Core delivery team

- **Solution Architect / AI Technical Lead** — shared, approximately 50%
- **AI Engineer — my role** — full-time
- **Second AI/Data Engineer** — full-time
- **Backend Engineer** — approximately 75–100%
- **QA Engineer** — approximately 50%

Shared roles

- **Business Analyst / Domain Analyst** — approximately 40–50%
- **DevOps/Cloud Engineer** — approximately 20–30%
- **Project Manager / Delivery Lead** — approximately 20–30%

This meant the nominal headcount could look like seven or eight people, but the effective engineering capacity was closer to four-and-a-half to five full-time equivalents.

My role

I was one of the core AI engineers. My main ownership was:

- document ingestion and chunking design;
- metadata strategy;
- dense retrieval with MiniLM + FAISS;
- BM25 lexical retrieval;
- RRF-based hybrid retrieval;
- retrieval diagnostics and evaluation;
- Neo4j enrichment integration with the AI pipeline;
- construction of the grounded evidence boundary;
- prompt orchestration with versioned templates;
- provider-independent LLM integration with the team;
- provenance validation;
- debugging retrieval/generation failures;
- presenting retrieval-quality findings in internal and client reviews.

The backend engineer primarily owned FastAPI delivery and helped with PostgreSQL integration. The architect owned the overall solution direction and reviewed major design decisions. The second AI engineer

supported data preparation, graph modeling, prompt experiments, test fixtures, and evaluation. We collaborated heavily rather than operating as isolated component owners.

Client stakeholders

The client side typically included:

- **Product Owner / Engineering Manager** — business sponsor for the POC
- **OMS Solution Architect** — architecture and system-dependency SME
- **Senior OMS Engineer / SDD SME** — explained existing design practices
- **Platform/API SME** — clarified Order/Payment interfaces
- **Security representative** — joined specific reviews for data/logging concerns

The product owner and architect attended milestone demos. Domain SMEs joined discovery and validation sessions.

Internal stakeholders

Our side included:

- delivery/project manager;
- solution architect/AI lead;
- AI engineers;
- backend engineer;
- QA;
- shared DevOps engineer;
- business analyst.

The architect and PM handled executive status communication; engineers handled technical working sessions.

---

2. How the project started

The project started with a client discussion around **design-document turnaround time**.

The client had a fairly mature engineering process. Before implementing a change, engineers were expected to produce design artifacts explaining affected services, APIs, error handling, persistence, downstream events, and non-functional considerations. The problem was not that these documents did not exist. In fact, the client had many years of documentation. The problem was finding and reconciling the right information quickly.

For a change such as partial refunds, an architect might have to examine:

- the current Order Management architecture;
- the Order Service ownership model;
- Payment Service contracts;
- gateway failure behavior;
- historical order-cancellation SDDs;

- event contracts;
- business requirements;
- security and audit rules.

The first draft could take several hours to more than a day depending on familiarity with the domain. The client was also concerned that knowledge was concentrated in a few senior engineers.

During an early steering discussion, the client asked whether an AI assistant could generate an SDD automatically. Our architect pushed back on the phrase “automatically” and reframed the problem:

> The first goal should not be autonomous design. The first goal should be a grounded, traceable draft that reduces discovery effort and gives an architect a better starting point.

That became the POC hypothesis.

The client did not want a large implementation commitment before seeing evidence that enterprise retrieval and grounding would work. We therefore proposed a POC instead of immediately building a production platform.

The POC would answer four questions:

1. Can the system find the correct enterprise evidence for a requirement?
2. Can it combine unstructured document evidence with explicit system relationships?
3. Can an LLM generate useful design content without inventing systems/APIs?
4. Can the output be traced back to the evidence used?

---

3. Discovery and requirement gathering — Weeks 1–2

The first two weeks were mostly discovery rather than coding.

We held several working sessions rather than one large requirements meeting.

First discovery call

The product owner explained the current process. Engineers received requirements from product teams, reviewed existing documents, spoke with service owners, and prepared SDDs manually.

The most time-consuming step was usually not writing English. It was answering questions like:

- Which service actually owns this decision?
- What existing API should be reused?
- What happens after the payment operation succeeds?
- Is there already a failure-handling pattern?
- Which downstream systems could be affected?
- Which design statements are assumptions versus documented facts?

This was important because it changed our mental model. The POC was not primarily a “document-generation” project. It was an **enterprise evidence-retrieval and grounding project with generation at the end**.

Domain workshop

The client gave us an anonymized feature scenario: support item-level partial refunds for a completed order.

We deliberately used a bounded domain for the POC.

The sample knowledge set represented:

- a partial-refund requirement;
- OMS architecture;
- refund interface documentation;
- an older order-cancellation SDD.

The client SME walked us through the expected ownership boundaries:

```
Order Service
→ determines eligibility
→ orchestrates the refund

Payment Service
→ executes the financial refund

Payment Gateway
→ external provider

Order Service
→ records the outcome
→ publishes OrderItemsRefunded

Inventory + Notification
→ consume downstream refund information
```

We also captured requirements such as:

- only COMPLETED orders are eligible;
- shipping is not refunded in the first release;
- cumulative quantity/value cannot exceed the original purchase/capture;
- idempotency is required;
- payment timeout is an ambiguous state and requires reconciliation;
- audit information must be retained;
- sensitive information must not leak into logs/events.

Functional requirements for the POC

By the end of discovery, we agreed on a narrow functional scope.

The system had to:

1. accept a requirement and requested SDD section;
2. retrieve relevant document evidence;

3. preserve source/chunk provenance;
4. identify known system entities;
5. retrieve bounded architecture relationships;
6. build a grounded prompt;
7. generate an impact-analysis draft;
8. validate source/chunk citations;
9. persist the request and generation attempt;
10. expose the workflow through an API.

Non-functional requirements

Because this was a POC, non-functional requirements were lighter than production but still meaningful.

We agreed that:

- the system should be explainable enough to debug retrieval;
- provider/model changes should not require rewriting retrieval;
- normal tests must not call paid LLM APIs;
- graph failure should not cause invented relationships;
- logs should not contain full sensitive documents or credentials;
- evidence provenance should be preserved;
- latency should be interactive, although no hard production SLA was committed.

Initial POC success criteria

The client and our architect agreed on practical success criteria rather than one headline accuracy number.

The POC would be considered useful if:

- relevant evidence appeared consistently in a small manually labeled test set;
- exact API/identifier questions worked, not just natural-language paraphrases;
- generated output referenced supplied evidence;
- an OMS SME considered the draft useful as a starting point;
- the architecture demonstrated how the solution could evolve into a controlled enterprise service.

Explicitly out of scope

We deliberately excluded:

- enterprise-wide Confluence/SharePoint connectors;
- PDF/OCR support;
- production authentication/RBAC;
- tenant isolation;
- distributed vector infrastructure;
- high availability;
- disaster recovery;
- automated production deployment;
- autonomous SDD approval;
- every SDD section;

- large-scale traffic;
- production Bedrock/Kendra deployment;
- full semantic fact-checking.

This scope discipline was important. Without it, the POC would have become an infrastructure program.

My involvement during discovery

I joined the technical workshops rather than waiting for final requirements.

My main questions were around evidence:

- What document contains the source of truth for refund eligibility?
- Are API identifiers important enough that exact matching is required?
- How often do existing SDDs contain reusable patterns?
- When two sources disagree, which one wins?
- Does “related service” mean “must change,” or only “needs review”?
- What evidence should a generated statement cite?

Those questions later directly influenced metadata, BM25, graph enrichment, and validation.

---

4. Initial solution discussion

At the end of discovery we held an architecture workshop.

The first temptation was to use a managed search/GenAI service end-to-end. A managed option would have been faster to stand up, but it would make retrieval behavior less transparent during the POC.

We considered three broad approaches.

Option 1 — LLM with large documents directly in the prompt

This was rejected quickly.

It would have been simple:

```
requirement
+ all documents
→ LLM
```

But it would be expensive, difficult to scale, and hard to explain when the model missed or invented information.

Option 2 — Managed enterprise retrieval

We discussed managed search/retrieval services because the historical enterprise architecture had references to services such as Kendra.

The benefit would be faster infrastructure setup and enterprise search capabilities.

The concern for this POC was that the client specifically wanted to understand whether **retrieval quality and grounding** were viable. We wanted control over:

- chunking;
- embeddings;
- lexical search;
- fusion;
- graph enrichment;
- evaluation.

We therefore kept managed retrieval as a production option rather than the POC core.

Option 3 — Custom retrieval + graph + LLM

This became the POC architecture.

```
documents
→ custom ingestion/chunking
→ dense retrieval
+ lexical retrieval
→ fusion
→ graph enrichment
→ grounded context
→ LLM
→ validation
```

The architect liked this because each step could be measured independently.

Technology choices

Python

The AI stack, FAISS, Sentence Transformers, Neo4j drivers, provider SDKs, and evaluation libraries were all convenient in Python.

Sentence Transformers + FAISS

We chose `all-MiniLM-L6-v2` with FAISS as a lightweight local semantic baseline.

For a tiny POC corpus, a distributed vector database would have been unnecessary.

BM25

We initially planned dense retrieval as the baseline, but we kept lexical search on the architecture backlog because API identifiers and error codes were clearly important.

Neo4j

The client cared about service impact, not just document similarity. Neo4j gave us an explicit relationship layer.

PostgreSQL

We needed transactional workflow state, prompt versions, generation requests, and provider attempts. Neo4j was not appropriate for that.

FastAPI

The backend was Python-heavy, and FastAPI gave us typed APIs with low overhead.

Hosted LLM provider abstraction

We did not want the POC to be coupled to one provider. Generation was put behind a generic interface.

---

5. Project plan

After architecture sign-off, the PM converted the plan into five two-week sprints.

The POC had three formal client milestones:

- **Architecture checkpoint — end of Week 2**
- **Mid-POC technical demo — end of Week 6**
- **Final POC demo — end of Week 12**

We also had weekly status calls and a short internal demo at the end of most sprints.

---

6. Sprint 1 — Weeks 3–4: ingestion and dense retrieval baseline

The first engineering sprint focused on making the document pipeline deterministic.

The goal was not to generate an SDD yet. The goal was:

> Given a query, can we reliably retrieve the correct section of the client's sample documents?

Ingestion work

The sample corpus contained requirements, architecture, interface documentation, and an existing cancellation SDD.

I worked on the ingestion contract with the second AI engineer.

We introduced a common `Document` representation so future source-specific loaders would not leak into retrieval.

The pipeline became:

```
source file
→ loader
→ normalization
→ metadata extraction
→ section-aware chunking
→ DocumentChunk
```

First setback — naive chunking

Our first internal version was too simplistic. It treated some front matter and small headings as independent retrievable units.

That created chunks that were technically valid but semantically poor.

Problem

A query could retrieve metadata like:

```
Requirement ID: OMS-REF-001
Owner: Order Platform
```

without the actual business rule.

Investigation

We inspected retrieved chunks manually and noticed that title/front-matter blocks were occupying retrieval positions.

Root cause

The parser did not distinguish between provenance metadata and meaningful evidence.

Fix

We:

- retained front matter as metadata;
- stopped emitting metadata-only chunks;
- preserved section heading + body together;
- used overlap only when an oversized section had to be split.

Result

The chunk set became more meaningful and easier to debug.

The final reconstruction used approximately:

```
1,200-character max chunk size
120-character overlap for oversized sections
```

Dense retrieval

I then implemented the semantic baseline using:

```
sentence-transformers/all-MiniLM-L6-v2
384 dimensions
FAISS IndexFlatIP
```

We L2-normalized vectors so inner product acted like cosine similarity.

The dense baseline worked reasonably well for natural-language paraphrases.

For example, a query like:

> What should happen when the external financial provider does not respond?

could retrieve Payment Gateway timeout material even when the document used different wording.

Sprint 1 review

At the internal review, the architect was satisfied with semantic retrieval but asked us to test less friendly enterprise queries.

The client SME gave us several examples containing literal identifiers and endpoint names.

That feedback exposed the next problem.

---

7. Sprint 2 — Weeks 5–6: BM25, hybrid retrieval, and Neo4j enrichment

The second sprint changed the retrieval architecture significantly.

Dense retrieval failed on exact identifiers

One of the test queries was:

```
OMS-REF-001
```

Dense retrieval was weak because the string itself carries little semantic meaning.

Another was:

```
IDEMPOTENCY_CONFLICT
```

The client made an important point:

> In enterprise engineering, exact strings are not edge cases. API paths, error codes, event names, and requirement IDs are part of normal work.

That made a lexical retriever necessary.

BM25 implementation

I added BM25 as a parallel retrieval path.

The tokenizer preserved technical strings and expanded useful components while keeping exact identifiers available.

The retrieval architecture became:

```
query
→ dense top 10
+
BM25 top 10
→ fusion
```

Why RRF

We could not safely add FAISS similarity and BM25 scores because they are on different scales.

We therefore used Reciprocal Rank Fusion:

```
RRF = Σ 1 / (60 + rank)
```

This combined rankings rather than raw scores.

Mid-POC retrieval demo

By the end of Week 6, we had three observable modes:

- dense;
- lexical;
- hybrid.

For the client demo we showed queries such as:

- Who owns refund eligibility?
- What API should support use?

- What happens during a gateway timeout?
- What is `IDEMPOTENCY_CONFLICT`?
- What event is published after success?

This was the first point where the client could see that retrieval quality was becoming a measurable engineering problem rather than an LLM magic trick.

Adding Neo4j

At the same time, we began graph enrichment.

The client architect had raised an important concern:

> A document may mention Payment Service, but the impact analysis also needs to show how it connects to other components.

The graph contained 12 canonical entities and 20 relationships.

Examples:

```
Support Portal CALLS Order Service
Order Service CALLS Payment Service
Payment Service CALLS Payment Gateway
Order Service PUBLISHES OrderItemsRefunded
Inventory Service CONSUMES OrderItemsRefunded
Notification Service CONSUMES OrderItemsRefunded
```

Why deterministic entity matching

For the POC graph, entity names were known.

We therefore did not use an LLM for entity extraction.

I helped implement deterministic canonical-name matching over the retrieved evidence.

This was cheaper, repeatable, and easier to test.

Graph traversal setback

During development we experimented with one-hop and two-hop traversal.

Two-hop results quickly became noisy.

A Payment Service query could bring in many adjacent components that were related but not necessarily impacted.

Problem

Graph enrichment created too much contextual evidence.

Root cause

Relationship existence was being treated too generously as design impact.

Fix

We:

- made traversal bounded;
- used one hop as the default;
- enforced a maximum depth;
- kept graph evidence separate from document evidence;
- changed prompt language so adjacency meant “known relationship,” not “definite modification.”

Result

The graph became supporting evidence instead of an uncontrolled expansion mechanism.

---

8. Sprint 3 — Weeks 7–8: grounded context, prompts, PostgreSQL, and LLM generation

By Week 7 the system could retrieve good candidate evidence and graph relationships.

The next challenge was transferring that evidence safely into generation.

GroundedContext

We did not want the generation service to receive FAISS objects, BM25 scores, and Neo4j records directly.

We introduced a stable evidence boundary:

```
GroundedContext
```

It contained:

- requirement;
- document evidence;
- graph evidence;
- provenance;
- warnings.

This became one of the most useful design decisions in the POC because it separated:

```
evidence acquisition
```

from:

```
generation
```

The LLM layer no longer cared how evidence was found.

PostgreSQL

We introduced PostgreSQL for workflow state.

The main tables were:

```
prompt_templates
generation_requests
generation_runs
```

Prompt templates were versioned because the client wanted generated output to be auditable.

A generation request stored the requirement and workflow status.

Generation runs stored provider/model attempts, output or error, latency, usage, and prompt version.

Prompt design

The prompt explicitly required the model to:

- use supplied evidence only;
- treat retrieved content as data, not instructions;
- distinguish documented facts from possible impacts;
- avoid inventing APIs/systems;
- state when evidence was insufficient.

The generated output was intended to be an architect-reviewed draft, not an autonomous decision.

Provider abstraction

We implemented a generic `LLMClient`.

That allowed the higher layers to remain independent of OpenAI/Gemini SDKs.

The flow became:

```
PreparedGenerationRequest
→ LLMClient
→ provider adapter
```

Generation setback — hallucinated citation

One early generated response contained a citation-like chunk ID that was not actually in the supplied context.

Our first validator was not strict enough.

Problem

The response looked grounded but referenced an unknown chunk.

Investigation

We compared the generated citation against the `GroundedContext`.

Root cause

Validation was looking for citation-shaped text but did not enforce exact membership strongly enough.

Fix

We tightened deterministic provenance validation so every explicit source path and chunk ID had to exist in the supplied evidence.

Result

Unknown citations caused validation failure instead of silently passing.

This became an important client talking point:

> We cannot prove factual correctness with a citation validator, but we can at least prevent the system from citing evidence it never received.

---

9. Sprint 4 — Weeks 9–10: FastAPI and production-style reliability

Once generation worked end-to-end, the backend engineer integrated the workflow into FastAPI.

I worked with him on dependency boundaries and failure behavior.

API flow

The main endpoint became:

```
POST /api/v1/generations
```

The request flow was:

```
Pydantic validation
→ request ID
→ generation request persisted
→ hybrid retrieval
→ entity matching
→ Neo4j
→ GroundedContext
→ prompt
→ LLM
→ validation
→ persistence
→ response
```

Routes remained thin. The application service owned orchestration.

Shared resources

We made the embedding model, FAISS index, BM25 retriever, graph driver, and provider clients process-level resources rather than recreating them per request.

Database sessions remained request-scoped.

Operational hardening

This sprint added:

- structured JSON logging;
- `X-Request-ID` correlation;
- stage latency measurements;
- retries with backoff;
- provider fallback;
- lightweight retrieval cache;
- `/health`;
- `/ready`;
- safe error mapping;
- environment-based secrets/configuration.

Provider failure issue

Our primary OpenAI configuration began returning quota/rate-related errors.

Instead of blocking the POC, the provider abstraction allowed us to fall back to Gemini.

Problem

Primary provider calls failed.

Investigation

Logs showed 429-style provider failures.

Root cause

The configured account did not have usable quota for those calls.

Fix

The generation service exhausted eligible retry policy and moved to the configured Gemini provider.

Result

The same prepared context could be generated without rebuilding retrieval or graph evidence.

The test also revealed a design improvement: permanent quota exhaustion should ideally be classified differently from temporary rate limiting so the system does not waste time retrying a condition that will not recover quickly.

Neo4j graceful degradation test

We deliberately stopped Neo4j and sent a generation request.

The intended policy was:

```
retrieval available
graph unavailable
→ continue document-only
```

This worked.

The logs showed zero graph evidence, but document evidence remained and generation could continue.

However, the failure took too long.

Problem

Graceful degradation was logically correct but slow.

Investigation

We inspected stage latency.

Root cause

Neo4j driver retry behavior consumed roughly a minute before giving up in one manual test.

Fix during POC

We documented the issue and maintained bounded configuration.

Production recommendation

Use a much tighter retry budget, explicit deadlines, and a circuit breaker.

This was a useful lesson: a fallback mechanism is not production-ready merely because it eventually succeeds.

Retrieval cache

We also demonstrated the in-process retrieval cache.

First identical request:

```
cache miss
```

Second:

```
cache hit
```

The cached retrieval stage dropped to effectively zero milliseconds.

For the POC this was sufficient. We explicitly noted that a multi-instance production system would require a version-aware distributed strategy if shared caching mattered.

---

10. Sprint 5 — Weeks 11–12: evaluation, reranking experiment, and final demo

The final sprint was intentionally not “add more features.”

The architect wanted evidence that the POC worked.

We built a manually labeled retrieval dataset containing 20 queries.

The categories included:

- semantic paraphrases;
- exact identifiers;
- API questions;
- business rules;
- ownership;
- failure handling;
- event-driven behavior;
- architecture relationships;
- security/audit;
- multi-evidence questions.

Each case had known relevant chunk IDs.

Metrics

We measured:

- Precision@K;
- Recall@K;
- Hit Rate@K;
- MRR.

At:

```
K = 1, 3, 5, 10
```

Unexpected result

The results challenged our assumption that hybrid retrieval would automatically be best.

Baseline MRR@10:

```
Dense   0.559
BM25    0.825
Hybrid  0.721
```

BM25 was the strongest first-rank retriever on this small domain.

That made sense after error analysis because many client queries contained explicit technical strings.

However, hybrid had:

```
Recall@10 = 0.942
Hit@10    = 1.000
```

That told us something different:

> Hybrid was very good at getting relevant evidence into the candidate set, but not always good at ordering it.

Reranker experiment

I proposed testing a CrossEncoder only after seeing those metrics.

We used:

```
cross-encoder/ms-marco-MiniLM-L-6-v2
```

on the top 10 hybrid candidates.

Results:

```
Hybrid MRR@10           0.721
Hybrid + reranker       0.831

Recall@5
0.783 → 0.858

Hit@5
0.950 → 1.000
```

Several difficult cases improved:

```
gateway timeout       5 → 3
OMS-REF-001           7 → 5
support API            4 → 3
shipping rule          2 → 1
transactional outbox   2 → 1
```

But one notification query regressed from rank 2 to rank 4.

Latency tradeoff

The quality improvement came with a large latency cost.

```
Hybrid retrieval        ≈ 10 ms
Reranked retrieval      ≈ 426 ms
```

We therefore did **not** quietly wire the CrossEncoder into the FastAPI production path.

We presented it as an evaluated optional mode.

That decision was important for credibility:

> The POC showed the reranker improves aggregate ranking, but we would need a larger representative query set and an agreed latency budget before enabling it by default.

Generation evaluation

We prepared 10 generation-evaluation cases.

The formal evaluator tracked:

- structural validity;
- provenance validity;

- expected-fact coverage;
- human relevance;
- unsupported claims;
- latency;
- tokens.

Only one genuine saved result had been fully evaluated in the reconstruction.

We did not claim 100% generation quality from that.

During demos we qualitatively reviewed generated drafts, but the formal generation-quality dataset was still immature compared with retrieval evaluation.

This became one of the limitations presented to the client.

---

11. How the team worked day to day

The project used a lightweight Agile process rather than heavy ceremony.

Daily stand-up

The core delivery team had a 15-minute internal stand-up.

Typical discussion:

- what was completed yesterday;
- current technical blocker;
- whether client input was needed;
- upcoming demo risk.

Sprint planning

At the start of each two-week sprint, the PM and architect reviewed the POC backlog.

Technical tickets were created for work such as:

```
Implement section-aware chunking
Persist FAISS chunk mapping
Add BM25 retriever
Implement RRF fusion
Seed OMS graph
Create GroundedContext
Version prompt templates
Add provider fallback
Add provenance validator
Add retrieval evaluation CLI
```

Git workflow

We used feature branches and pull requests.

Examples:

```
feature/hybrid-retrieval
feature/neo4j-enrichment
feature/generation-validation
```

At least one peer reviewed important AI pipeline changes.

The architect reviewed changes affecting system boundaries rather than every code-level change.

Testing

Most tests were offline and deterministic.

We did not allow normal `pytest` to call paid model providers.

Unit tests covered:

- chunking;
- metadata;
- retrieval;
- fusion;
- graph matching;
- validation;
- database behavior;
- API schemas;
- evaluation metrics;
- mocked reranker behavior.

By the end of the POC the reconstructed project had:

```
124 passed
3 skipped
```

Client interaction

The client did not attend daily stand-ups.

They were involved through:

- weekly status calls;
- discovery workshops;
- architecture checkpoint;
- mid-POC retrieval demo;
- final POC demo;

- ad hoc SME clarifications.

This kept the engineering team productive while still giving the client regular control points.

---

12. How client feedback changed the solution

The POC changed several times because of feedback.

Feedback example 1 — exact enterprise strings matter

Client feedback

The OMS engineer said a system that understands “refund” but misses an exact API or error code would not be useful for design work.

Engineering change

We added BM25 and technical-token handling.

Result

`IDEMPOTENCY_CONFLICT` ranked first lexically, and `OMS-REF-001`—which dense retrieval missed in top 10—ranked first with BM25.

Feedback example 2 — relationships are not the same as impact

Client feedback

The solution architect objected to early graph-driven wording that made every neighboring service look impacted.

Engineering change

We bounded graph traversal and changed context/prompt semantics.

Result

Graph edges became supporting relationship evidence rather than change assertions.

Feedback example 3 — show me where the answer came from

Client feedback

A business demo participant asked whether the generated statement could be traced back to the original design material.

Engineering change

We strengthened source/chunk provenance and deterministic citation validation.

Result

Unknown source/chunk citations caused validation failure.

Feedback example 4 — what if the graph is unavailable?

Client feedback

The engineering manager asked whether the entire platform would stop if Neo4j was temporarily unavailable.

Engineering change

We implemented document-only graph degradation.

Result

Generation remained possible from retrieved documents, with a warning that graph evidence was unavailable.

Feedback example 5 — can you prove hybrid is better?

Client feedback

During a technical review, the client architect asked for measured retrieval results rather than qualitative screenshots.

Engineering change

We created a 20-query manually labeled evaluation set.

Result

The data showed BM25 actually beat baseline hybrid MRR, which forced us to refine our architecture story instead of claiming hybrid was universally superior.

---

13. Architecture evolution

The final architecture was not designed perfectly on Day 1.

It evolved through evidence.

Version 0 — feasibility concept

```
Requirement
→ retrieve documents
→ LLM
```

Useful for discussing the idea, but too vague.

Version 1 — dense RAG baseline

```
Documents
→ section chunking
→ MiniLM embeddings
→ FAISS
→ LLM
```

This proved semantic retrieval but exposed exact-identifier failures.

Version 2 — hybrid retrieval

```
FAISS
+
BM25
→ RRF
```

This improved candidate coverage and exact-match behavior.

Version 3 — graph-enriched RAG

```
Hybrid retrieval
→ entity matching
→ Neo4j
→ document + graph evidence
```

This added explicit service relationships.

Version 4 — grounded generation platform

```
Hybrid retrieval
→ Neo4j
→ GroundedContext
→ versioned PostgreSQL prompt
→ provider-independent LLM
→ provenance validation
```

This turned the retrieval prototype into a controlled generation workflow.

Version 5 — API and reliability

```
FastAPI
→ structured logging
→ request IDs
→ retries/fallback
→ cache
→ readiness/liveness
→ graceful graph degradation
```

This made the POC operable like a service.

Version 6 — measured retrieval + optional reranker

Evaluation showed:

```
Hybrid candidate recall = strong
Hybrid ordering = weaker
```

so we tested:

```
Hybrid top 10
→ CrossEncoder
→ reranked results
```

We kept it optional because latency increased substantially.

---

14. My personal contribution

I would describe my contribution as substantial but not solitary.

What I owned directly

I owned most of the retrieval-quality path end-to-end:

```
document representation
→ chunking/metadata
→ embeddings
→ FAISS
→ BM25
→ RRF
```

```
→ retrieval diagnostics
→ evaluation
```

I also contributed strongly to:

- deterministic entity matching;
- integration between retrieval and Neo4j enrichment;
- `GroundedContext`;
- prompt evidence formatting;
- provider abstraction design;
- provenance validation;
- failure analysis;
- reranker evaluation.

What I coded

Examples of modules I worked on or co-owned included:

```
ingestion loaders
normalization
metadata extraction
chunker
embedding service
FAISS vector store
BM25 lexical retriever
RRF fusion
retrieval coordinator
entity matcher
context builder
prompt builder
generation validation
retrieval evaluation
reranking experiment
```

What I did not own alone

The backend engineer took primary ownership of:

- FastAPI route structure;
- API schemas;
- request/session dependency lifecycle;
- API error mapping.

PostgreSQL schema and generation persistence were collaborative between the backend engineer, architect, and AI team.

The architect owned final architectural sign-off.

The second AI engineer helped with:

- corpus preparation;
- graph schema/seed;
- tests;
- prompt experiments;
- evaluation labels.

The QA engineer helped turn expected behavior into repeatable regression cases.

Problems I personally debugged

The strongest examples are:

1. dense retrieval failing on exact technical identifiers;
2. metadata/front matter becoming poor retrieval chunks;
3. hybrid RRF lowering some strong lexical ranks;
4. hallucinated/unknown chunk citation;
5. reranker quality-versus-latency tradeoff;
6. graph enrichment generating excessive relationship noise;
7. provider failure/fallback behavior.

What I presented

In client demos I would primarily present:

- how the requirement becomes retrieval queries;
- dense vs lexical behavior;
- the evidence returned;
- why graph relationships were added;
- retrieval evaluation results;
- failure analysis;
- reranking tradeoff;
- limitations and next-step recommendations.

The architect would frame the overall solution and production roadmap.

---

15. Team collaboration

The project worked because responsibilities overlapped at the boundaries.

Business Analyst / domain SME → AI engineers

The BA converted workshop notes into functional statements and clarified terminology.

We then mapped those concepts to document metadata, test questions, and prompt requirements.

AI engineer ↔ client OMS SME

This collaboration was critical for relevance labels.

A technically plausible chunk was not automatically a correct answer.

The client SME helped define which evidence actually answered a question.

AI engineer ↔ backend engineer

We agreed on contracts rather than sharing internal objects.

For example:

```
GenerationApplicationService
```

did not expose FAISS internals to FastAPI.

AI engineer ↔ architect

The architect challenged decisions such as:

- why Neo4j was necessary;
- why raw FAISS/BM25 scores were not added;
- whether graph failure should be fatal;
- why the reranker was not enabled by default.

Those reviews improved the design.

DevOps engineer

The DevOps engineer helped with:

- Dockerized PostgreSQL/Neo4j setup;
- environment conventions;
- service startup;
- secrets handling;
- basic deployment-readiness discussion.

Full cloud deployment remained out of POC scope.

---

16. Evaluation strategy in detail

The evaluation effort was intentionally split into **retrieval** and **generation**.

Retrieval evaluation

A manually labeled gold set contained 20 queries.

The AI team proposed the queries; the domain SME/architect reviewed the expected evidence for representative cases.

The test set intentionally mixed query styles.

Examples:

```
semantic:
"What should happen when the external financial provider does not respond?"

identifier:
"OMS-REF-001"

API:
"What API should customer support use for a partial refund?"

business rule:
"Is shipping refunded in the first release?"

architecture:
"Which service owns refund eligibility and which owns financial execution?"
```

Metrics:

```
Precision@K
Recall@K
Hit Rate@K
MRR
```

This was much more useful than one vague “accuracy” score.

Baseline results

```
Dense MRR@10   = 0.559
BM25 MRR@10    = 0.825
Hybrid MRR@10  = 0.721
```

Hybrid:

```
Recall@10 = 0.942
Hit@10 = 1.000
```

Reranked results

```
MRR@10      0.721 → 0.831
Recall@5    0.783 → 0.858
```

```
Hit@5       0.950 → 1.000
```

Latency:

```
Hybrid ≈ 10 ms
Reranked ≈ 426 ms
```

Generation evaluation

We prepared 10 generation cases.

The dimensions were:

- structural validity;
- provenance validity;
- expected-fact coverage;
- human relevance;
- unsupported claims;
- latency;
- tokens.

At the end of the reconstructed POC, only one saved genuine generation result had been formally scored end-to-end.

We therefore presented generation evaluation as an area requiring expansion before production.

Human validation

The client SME reviewed representative retrieval and generated-output examples during demos.

We never positioned automatic metrics as a substitute for SME judgment.

---

17. Business-level POC outcome

The final POC was considered technically successful because it demonstrated the complete controlled workflow:

```
requirement
→ evidence retrieval
→ relationship enrichment
→ grounded generation
→ validation
→ persistence
→ API response
```

It also produced measurable retrieval evidence rather than only demo screenshots.

For a small set of pilot design exercises, the team estimated that the assistant could reduce **first-draft/manual documentation effort by roughly 30%** and improve **turnaround by roughly 15%** compared with the manual baseline.

Those figures were presented as **directional POC estimates**, not production KPIs. They reflected the reduction in document-searching and first-draft preparation time on the narrow scenario, not end-to-end software delivery improvement across the client organization.

---

18. Final client demo — Week 12

The final demo was structured around one realistic requirement rather than a collection of disconnected features.

The requirement was:

> Enable partial refunds for individual items of a completed order.

Demo flow

1. Requirement submission

We submitted the requirement through the FastAPI generation endpoint.

The client saw the generated request ID and status.

2. Evidence retrieval

We showed the top retrieved evidence.

Examples included:

- completed-order eligibility;
- Payment Service refund interface;
- gateway timeout behavior;
- cancellation SDD patterns;
- audit/security requirements.

3. Exact-search example

We separately showed `OMS-REF-001` and `IDEMPOTENCY_CONFLICT` to demonstrate why lexical retrieval existed.

4. Architecture relationships

We showed Neo4j relationships such as:

```
Order Service → Payment Service → Payment Gateway
```

and:

```
Order Service → OrderItemsRefunded → Inventory/Notification
```

We explicitly explained that a graph edge indicates a relationship, not guaranteed implementation impact.

5. Grounded generation

The system generated the impact-analysis draft using the supplied document and graph evidence.

6. Provenance

We showed source/chunk references and explained validation.

7. Failure behavior

Rather than doing a risky live outage demo, we showed logs/results from graph-degradation and provider-fallback testing.

8. Evaluation results

The most important slide was the retrieval comparison.

```
Dense MRR      0.559
BM25 MRR       0.825
Hybrid MRR     0.721
Reranked MRR   0.831
```

We explained why BM25 was unexpectedly strong and why CrossEncoder was still not automatically enabled.

Client questions

The client asked questions such as:

- How would this work with Confluence and SharePoint?
- Can access permissions be applied before retrieval?
- What happens when documents contradict one another?
- Why use Neo4j instead of metadata?
- Can this scale beyond a local FAISS index?
- Can we deploy on AWS?
- How would we control model cost?
- How do we know a citation really supports the claim?

We answered some directly and moved others into the production roadmap.

Limitations we openly presented

We explicitly stated:

- corpus was tiny;
- no production ACL/RBAC;
- no live enterprise connectors;
- no distributed retrieval;
- reranker had a latency cost;
- generation evaluation was not yet statistically meaningful;
- provenance validation did not prove factual entailment;
- graph retry/degradation latency needed improvement;
- synchronous generation would eventually need reconsideration.

That honesty helped the POC because the client could distinguish feasibility from production readiness.

---

19. POC-to-production recommendations

The POC should not be deployed unchanged.

What was actually built in the POC

```
custom document ingestion
section-aware chunking
metadata
MiniLM embeddings
FAISS
BM25
RRF hybrid retrieval
optional CrossEncoder evaluation
deterministic entity matching
Neo4j enrichment
PostgreSQL workflow/prompt persistence
GroundedContext
provider-independent LLM clients
OpenAI/Gemini generation path
deterministic provenance validation
FastAPI
structured logging
request IDs
retrieval cache
health/readiness
retry/fallback
graph degradation
offline retrieval evaluation
```

Recommended before production

1. Enterprise connectors

Add supported ingestion from:

- Confluence;
- SharePoint;
- PDF/DOCX repositories;
- object storage.

2. Incremental/versioned indexing

Replace full local rebuilds with:

```
source version/hash
→ changed document detection
→ reprocessing
→ versioned index
→ atomic promotion
```

3. Authorization

Access control must be applied before retrieval.

Tenant/document permissions should become metadata filters or index partitions.

4. Distributed search

Evaluate:

- OpenSearch;
- managed vector stores;
- distributed FAISS;
- hybrid enterprise-search services.

Use the same gold evaluation when migrating.

5. Async ingestion

A production AWS design could use:

```
S3 upload
→ event
→ Lambda/queue
→ ingestion/index refresh
```

Lambda was not part of the synchronous POC API.

6. Model/provider productionization

Add:

- circuit breakers;
- provider SLOs;
- cost limits;
- permanent-vs-transient error classification;
- model/prompt rollout controls.

7. Semantic validation

Add claim-level:

- entailment;
- contradiction checks;
- unsupported-claim detection;
- human approval workflow.

8. Observability

Move from basic structured logs to dashboards for:

- retrieval latency;
- provider latency;
- fallback rate;
- cache hit rate;
- validation failures;
- token cost;
- degraded graph requests.

9. Evaluation automation

Expand:

- retrieval gold set;
- generation gold set;
- multi-reviewer labels;
- category-level regression tests;
- production query sampling.

10. Security

Add:

- authentication;
- authorization;
- tenant isolation;
- secret management;
- PII handling;
- audit logs;
- prompt-injection controls.

11. Reliability

Add:

- high availability;
- backup/restore;
- distributed cache if required;
- index rollback;
- disaster-recovery runbooks.

12. Human feedback loop

A production workflow should support:

```
Prediction
→ validation
→ human/ground-truth feedback
→ error analysis
→ controlled prompt/taxonomy/retrieval change
→ re-evaluation
→ versioned release
```

---

20. Final timeline

Week / Sprint	Major Work	My Contribution	Deliverable / Client Milestone
Weeks 1–2 — Discovery	Client workshops, OMS partial-refund scenario, source-document review, POC scope, success criteria, architecture options	Asked evidence/retrieval questions, mapped document needs, helped define technical success criteria	Architecture checkpoint and agreed POC backlog
Sprint 1 — Weeks 3–4	Common document model, normalization, metadata, section-aware chunking, MiniLM embeddings, FAISS baseline	Co-owned ingestion/chunking; implemented dense retrieval and retrieval inspection	Internal dense-search demo
Sprint 2 — Weeks 5–6	BM25, tokenization, RRF hybrid retrieval, Neo4j seed/entity matching/bounded traversal	Implemented BM25/RRF path, debugged identifier misses, integrated retrieval with graph enrichment	Mid-POC client retrieval/graph demo
Sprint 3 — Weeks 7–8	GroundedContext, prompt templates, PostgreSQL request/run state, LLM provider abstraction, generation validation	Co-designed GroundedContext; worked on prompt evidence format,	First grounded impact-analysis draft

Week / Sprint	Major Work	My Contribution	Deliverable / Client Milestone
		provider abstraction, provenance validation	
Sprint 4 — Weeks 9–10	FastAPI integration, structured logging, request IDs, cache, retries/fallback, health/readiness, graceful graph degradation	Worked with backend on orchestration boundaries; debugged provider fallback, citation validation, Neo4j degradation	End-to-end service demo and reliability review
Sprint 5 — Weeks 11–12	20-query retrieval gold set, dense/BM25/hybrid comparison, CrossEncoder experiment, generation-eval framework, final documentation	Led retrieval evaluation/failure analysis, implemented reranker experiment, presented metrics/tradeoffs	Final client POC demo and production roadmap

21. Team structure

Role	Approx. Count	Responsibilities	My Interaction
Project Manager / Delivery Lead	1 shared	Scope, schedule, client status, risks, milestones	Weekly status, blockers, demo readiness
Solution Architect / AI Lead	1 shared	Overall architecture, technical reviews, client architecture discussions	Reviewed retrieval/graph/generation decisions with me
AI Engineer — me	1	Ingestion, retrieval, hybrid search, graph integration, grounding, validation, evaluation	Core technical contributor
AI/Data Engineer	1	Corpus prep, graph schema/seed, prompt/test support, evaluation labels	Daily collaboration and code reviews
Backend Engineer	1	FastAPI, API schemas, persistence integration, request lifecycle	Defined service contracts and integrated AI pipeline
QA Engineer	1 shared	Test scenarios, regression tests, defect tracking	Converted retrieval/generation behavior into repeatable tests
Business Analyst / Domain Analyst	1 shared	Requirement notes, domain terminology, client coordination	Clarified business rules and evaluation questions

Role	Approx. Count	Responsibilities	My Interaction
DevOps / Cloud Engineer	1 shared	Docker environments, config/secrets, deployment-readiness guidance	Environment setup and operational reviews

22. Final architecture journey

```
Version 0
Requirement → all/retrieved docs → LLM
Reason to change:
too little control and explainability
```

↓

```
Version 1
Section-aware chunks → MiniLM → FAISS
Reason to change:
semantic retrieval missed exact IDs/APIs
```

↓

```
Version 2
FAISS + BM25 → RRF hybrid
Reason to change:
text relevance did not explicitly model service dependencies
```

↓

```
Version 3
Hybrid retrieval → deterministic entity matching → Neo4j
Reason to change:
generation needed a stable, auditable evidence boundary
```

↓

```
Version 4
GroundedContext → versioned PostgreSQL prompts → provider-independent LLM
```

```
Reason to change:
needed provenance validation, workflow state, and API operation
```

↓

```
Version 5
FastAPI + persistence + logging + retries/fallback + cache +
health/readiness
Reason to change:
needed measured retrieval quality rather than demo intuition
```

↓

```
Final POC
Hybrid retrieval + graph enrichment + grounded generation + validation +
API
+
evaluation-only CrossEncoder reranker
```

The CrossEncoder remained optional because it improved ranking quality but increased retrieval latency from roughly 10 ms to roughly 426 ms.

---

23. Interview-ready narration A — 60–90 seconds

I worked on a POC for a retail client that wanted to reduce the manual effort involved in preparing Solution Design Documents. Their architects were spending significant time searching requirements, architecture documents, API contracts, and earlier SDDs before they could even start the design.

We built a grounded design-automation platform rather than an autonomous design generator. My main responsibility was the AI retrieval and grounding pipeline. I worked on section-aware ingestion and metadata, MiniLM embeddings with FAISS, BM25 lexical retrieval, and RRF-based hybrid search. We added Neo4j because document retrieval could find relevant text, but impact analysis also needed explicit relationships between services such as Order Service, Payment Service, and downstream events.

The retrieved document and graph evidence was converted into a `GroundedContext`, rendered using versioned prompts stored in PostgreSQL, and sent through a provider-independent LLM layer. We also added provenance validation, FastAPI integration, provider fallback, caching, logging, and graceful graph degradation.

The most useful part of the POC was evaluation. On a 20-query gold set, BM25 actually beat our initial hybrid ranking, while hybrid had excellent top-10 recall. That led us to test a CrossEncoder reranker, which improved MRR from 0.721 to 0.831 but added around 400 ms latency. So we kept reranking configurable rather than claiming it should always be enabled.

---

24. Interview-ready narration B — 3–4 minutes

The project started as a client POC for automating part of the Solution Design Document workflow. The client had a mature Order Management platform, but architects were spending a lot of time finding information across requirements, architecture docs, APIs, and historical SDDs. The idea was not to let an LLM design systems autonomously. We wanted to generate a grounded first draft with traceable evidence.

For the POC we used an item-level partial-refund requirement because it touched several realistic components—Order Service, Payment Service, an external gateway, inventory, notifications, events, and audit rules.

I was one of the core AI engineers. I mainly owned the ingestion and retrieval-quality path. We first normalized and section-chunked the source documents and preserved metadata such as source, section, and requirement IDs. Then we embedded chunks with `all-MiniLM-L6-v2` and stored them in FAISS.

The first issue we found was that semantic retrieval was good for paraphrases but poor for exact enterprise identifiers. For example, `OMS-REF-001` was missed by dense retrieval. So I added BM25 and combined dense and lexical rankings with Reciprocal Rank Fusion instead of adding raw scores.

The next requirement came from the client architect: an impact analysis needs more than similar text—it also needs system relationships. We added a small Neo4j knowledge graph and deterministic entity matching. We kept graph traversal bounded because two-hop traversal quickly produced unrelated neighbors, and we made it clear that adjacency was relationship evidence, not proof of implementation impact.

After retrieval and graph enrichment, we converted everything into a stable `GroundedContext`. That separated the retrieval infrastructure from generation. Prompt templates were versioned in PostgreSQL, and the generation service called providers through a generic `LLMClient`. We had OpenAI and Gemini adapters. We also added deterministic validation so the model could not cite source paths or chunk IDs that were not actually provided.

The backend engineer exposed the flow through FastAPI. We added request IDs, structured logging, caching, health/readiness, retries, provider fallback, and document-only degradation if Neo4j was unavailable.

The important part was that we evaluated retrieval instead of relying on demos. We created 20 manually labeled queries. Dense MRR was 0.559, BM25 was 0.825, and hybrid was 0.721. Hybrid still had 0.942 Recall@10 and 100% Hit@10, so the evidence was usually present but ordered poorly. That justified a CrossEncoder experiment. Reranking improved MRR to 0.831 and Hit@5 to 1.0, but increased latency from about 10 ms to about 426 ms, so we kept it optional.

The final client demo showed the partial-refund requirement going through retrieval, graph enrichment, grounded generation, validation, and persistence. We positioned the POC as successful feasibility, but we were explicit that production would still need enterprise connectors, ACL-aware retrieval, incremental indexing, distributed search, stronger semantic validation, and more representative generation evaluation.

---

25. Interview-ready narration C — deep dive with

natural interruption points

Opening — business problem

The project was a POC for a client that wanted to reduce the effort involved in preparing Solution Design Documents. Their engineers already had requirements, architecture docs, API contracts, and old SDDs, but discovery was manual and depended heavily on experienced architects.

Our proposal was to build a grounded assistant that could take a new requirement and prepare an evidence-backed impact-analysis draft.

[Likely interviewer interruption: “Why RAG instead of simply asking an LLM?”]

Because the useful information already existed in enterprise sources, and we needed traceability. Sending documents directly to an LLM would be difficult to scale and hard to evaluate. We wanted retrieval to be an explicit, measurable component.

Data and ingestion

We represented all source documents with a common internal `Document` model, normalized the content, extracted metadata, and used section-aware chunking.

We avoided fixed character windows where possible because design docs naturally contain meaningful headings such as APIs, failure handling, and security.

[Likely interruption: “What chunk size did you use and why?”]

The reconstruction used around 1,200 characters with 120-character overlap only when oversized sections had to be split. I would not claim that as a universal optimum; it was a practical POC setting.

Dense retrieval

We used `all-MiniLM-L6-v2`, which produces 384-dimensional embeddings, and FAISS `IndexFlatIP`.

We normalized query and corpus vectors, so inner product became equivalent to cosine similarity.

[Likely interruption: “Why FAISS? Would you use it at 10 million documents?”]

FAISS was ideal for a small explainable local POC. At large scale I would evaluate approximate indexes or a distributed/managed vector platform and preserve the same retrieval evaluation during migration.

BM25 and hybrid retrieval

Our first dense-only baseline exposed a weakness with exact enterprise strings.

`OMS-REF-001` is the best example. Dense retrieval did not place it in top 10, while BM25 ranked the correct requirement chunk first.

So we introduced BM25 and used RRF to fuse ranks.

[Likely interruption: “Why RRF instead of adding scores?”]

Because FAISS cosine similarity and BM25 relevance scores are not on the same calibrated scale. RRF combines rank positions instead.

Graph enrichment

The client architect then asked how the system would understand downstream dependencies.

We represented explicit OMS relationships in Neo4j.

Retrieved chunks were scanned for known entity names, and we performed bounded graph traversal.

[Likely interruption: “Why not ask the LLM to extract entities?”]

For a small fixed catalog, deterministic matching was cheaper, reproducible, and testable. I would consider entity linking when the catalog becomes open-ended or ambiguous.

[Likely interruption: “Does an adjacent node mean it is impacted?”]

No. That was actually an early problem. We changed the design so graph adjacency is only relationship evidence. The prompt must distinguish documented relationships from possible impact.

GroundedContext and generation

We converted document and graph evidence into `GroundedContext`.

That was the contract between retrieval and generation.

Versioned prompt templates were stored in PostgreSQL, and generation used a provider-neutral `LLMClient`.

[Likely interruption: “Why PostgreSQL when you already have Neo4j?”]

Neo4j stores topology. PostgreSQL stores transactional workflow state—prompt versions, generation requests, provider runs, status, tokens, and errors.

Validation

We added deterministic source/chunk validation after an early response invented a citation-like chunk ID.

[Likely interruption: “Does that prevent hallucination?”]

No. It prevents invented provenance. A model can still misinterpret a valid chunk. Provenance validation is not the same as factual entailment.

API and production concerns

FastAPI exposed the workflow. Expensive resources were initialized once per process, while database sessions were per request.

We added structured logging, correlation IDs, retries, fallback, cache, health/readiness, and graph degradation.

One manual failure test showed that Neo4j degradation worked but the driver spent roughly a minute retrying first.

[Likely interruption: “How would you fix that?”]

Tighter dependency deadlines, retry budgets, and a circuit breaker. A fallback that takes a minute is logically correct but operationally poor.

Provider fallback

OpenAI quota errors gave us a real provider-failure scenario.

The prepared context did not need to be rebuilt. The generation layer simply moved to Gemini after the eligible retry policy.

[Likely interruption: “Retry versus fallback?”]

Retry means same provider again. Fallback means switching providers.

Evaluation

The strongest part of the POC was retrieval evaluation.

We created 20 manually labeled queries across semantic, exact-identifier, API, business-rule, architecture, security, and multi-evidence categories.

Results:

```
Dense MRR@10   0.559
BM25 MRR@10    0.825
Hybrid MRR@10  0.721
```

This was surprising because hybrid was not best.

But hybrid had:

```
Recall@10 0.942
Hit@10    1.000
```

So candidate recall was excellent while ordering was weaker.

[Likely interruption: “What did you do about it?”]

We tested a CrossEncoder over the top 10 hybrid candidates.

MRR improved:

```
0.721 → 0.831
```

and Hit@5 improved:

```
0.950 → 1.000
```

but latency increased from roughly 10 ms to around 426 ms.

So we kept it optional.

[Likely interruption: “BM25 was 0.825. Why not use BM25 alone?”]

Because the current corpus is very literal and small. Dense retrieval still helped semantic cases, and hybrid had the best candidate recall. I would expand the evaluation set and decide using category-level performance, workload distribution, and latency rather than one metric.

Final outcome

The POC demonstrated that a controlled evidence pipeline could generate a traceable SDD draft and that retrieval quality could be measured and improved.

We considered the POC successful, but not production-ready.

Before production we recommended:

- enterprise source connectors;
- incremental/versioned indexing;
- ACL-aware retrieval;
- distributed search;
- better generation evaluation;
- semantic claim validation;
- asynchronous ingestion;
- production observability/security;
- model/prompt rollout controls.

That was the final distinction we maintained with the client:

```
POC proved feasibility.
Production requires governance and scale engineering.
```

26. The mental model to remember

If I had to reduce the entire project to one chain, it would be:

```
Business problem
→ architects spend time finding evidence
→ client asks for AI-assisted SDD drafting
→ POC scope is intentionally narrow
→ documents are normalized and section-chunked
→ dense retrieval works for meaning
→ exact identifiers fail
→ BM25 is added
→ RRF combines dense + lexical
→ client needs dependency context
→ Neo4j is added
→ graph becomes too broad
→ traversal is bounded
→ evidence is normalized into GroundedContext
→ prompts are versioned
→ LLM generation is provider-independent
→ hallucinated citations expose validation weakness
→ exact provenance validation is added
→ FastAPI turns it into a service
→ provider/graph failures expose reliability issues
→ retries, fallback, caching, degradation, observability are added
→ evaluation shows BM25 > hybrid MRR
→ hybrid still has excellent candidate recall
→ CrossEncoder is tested
→ ranking improves but latency increases
→ reranker remains optional
→ final POC proves feasibility
→ production roadmap focuses on ACLs, scale, indexing, governance, and
evaluation
```

If you can explain that chain naturally, you understand the project rather than memorizing isolated interview answers.